

معماری سیستم فاز اول Unixsee

داشبورد Next.js + لایه BFF با NestJS + پلتفرم عملیاتی WordPress

وضعیت: پیشنهادی

تاریخ: ۲۷ ژوئیه ۲۰۲۶

تصمیم معماری: Next.js فقط با NestJS ارتباط برقرار می‌کند. WordPress تجربه پنل مدیریت و فرایندهای کسب‌وکار/محتوا را فراهم می‌کند. NestJS همچنان درگاه اصلی برنامه، مرز مجزدهی، بک‌اند عملیاتی و سامانه بلادرنگ باقی می‌ماند.

۱. هدف و محدوده

این سند معماری پیشنهادی فاز اول اپلیکیشن Unixsee را بر اساس مسیرهای پیاده‌سازی شده فعلی در داشبورد و جریان‌های محصول مورد توافق تعریف می‌کند. تمرکز سند بر مرزهای سیستم، مالکیت داده، یکپارچه‌سازی‌ها، جریان‌های زمان اجرا و ترتیب پیاده‌سازی است، نه طراحی رابط کاربری یا جزئیات سطح پایین کد.

سورس فعلی Next.js این ماژول‌های داشبورد را نمایش می‌دهد:

- نمای کلی داشبورد شامل فهرست وب‌سایت‌ها، اعلان‌ها و فعالیت‌ها.
- پلن‌ها و یک فرم شبیه پرداخت که باید به جریان درخواست پلن تبدیل شود.
- فهرست وب‌سایت‌ها و جزئیات هر وب‌سایت.
- تیکت‌ها و گفت‌وگوهای تیکت.
- درخواست خدمات تکمیلی و پیگیری خدمات.
- پروفایل و تنظیمات امنیتی.
- موضوعات و مقالات مرکز راهنما.
- رابط دامنه‌ها که تا زمان تأیید صریح مالکیت و فرایندهای آن، باید خارج از محدوده قطعی بک‌اند فاز اول باقی بماند.

یادداشت درباره وضعیت سورس: رابط فعلی با داده‌های ثابت و آزمایشی کار می‌کند. این معماری مشخص می‌کند مدل‌های نمایشی فعلی چگونه باید توسط سرویس‌های واقعی تأمین شوند، بدون اینکه Next.js مستقیماً به WordPress یا سیستم‌های زیرساختی وابسته شود.

۲. جمع‌بندی معماری

Unixsee باید از یک معماری ماژولار و لایه‌ای با یک بک‌اند واحد در مقابل فرانت‌اند استفاده کند:

- Next.js لایه نمایش است.
 - NestJS نقش Backend-for-Frontend یا BFF، درگاه API، مرجع توکن و احراز هویت، مرز مجوزدهی، هماهنگ‌کننده فرایندها و درگاه بلادرنگ را دارد.
 - WordPress پلتفرم عملیات کسب‌وکار و محتوا، شامل تجربه مدیر سیستم، است.
 - PostgreSQL پایگاه داده مرجع برای وبسایت‌ها، سرورها، Agentها، متریک‌ها، عملیات‌ها و رویدادهای فعالیت است.
 - ذخیره‌سازی WordPress برای رکوردهای محتوایی و فرایندهای کسب‌وکاری سنگین از نظر مدیریت، هرجا اکوسیستم WordPress مزیت روشنی ایجاد کند، مرجع خواهد بود.
 - Edge Agentها فقط با NestJS ارتباط برقرار می‌کنند و هرگز مستقیماً با WordPress یا Next.js در تماس نیستند.
- استفاده از WordPress به‌عنوان پنل مدیریت به این معنا نیست که هر موجودیتی که در WordPress ویرایش می‌شود باید در پایگاه داده WordPress ذخیره شود. افزونه WordPress مربوط به Unixsee باید برای موجودیت‌های عملیاتی تحت مالکیت NestJS نقش رابط Control Plane را داشته باشد و در عین حال مالک بومی فرایندهای محتوایی و کسب‌وکاری مناسب WordPress باشد.

۳. معماری منطقی

```
مرورگر کاربر
|
| HTTPS / Socket.IO
v
Next.js داشبورد و رابط عمومی
|
| REST + Socket.IO فقط
v
NestJS BFF / هسته برنامه
|-- Tenant احراز هویت، توکن‌ها و مجوزدهی
|-- API تجمیع داشبورد و مدل‌های نمایشی
|-- ها، متریک‌ها، هشدارها و عملیات‌ها Agent، وبسایت‌ها، سرورها
|-- و ردپای ممیزی Unixsee فید فعالیت‌های
|-- WordPress یکپارچه‌سازی Adapter
|
+--> PostgreSQL / TimescaleDB در آینده
+--> Edge Agent ها از طریق HTTPS احراز شده
+--> WordPress Unixsee Bridge از طریق API خصوصی سرور به سرور
+--> Object Storage برای فایل‌های پیوست تیکت و درخواست

WordPress + Unixsee Bridge افزونه پنل مدیریت
|-- مدیریت کاربران و مشتریان
|-- درخواست پلن و فرایند راه‌اندازی
|-- تیکت‌ها و خدمات تکمیلی
|-- اعلان‌ها، مرکز راهنما، استوری‌ها، ارزیابی‌ها و عضویت‌ها
|-- در آینده: صورتحساب، پرداخت، نظرات مشتریان و وبلاگ
+--> WordPress پایگاه داده
+--> برای موجودیت‌های عملیاتی NestJS مدیریتی API
```

۳.۱. Control Plane در برابر Execution Plane

لایه	مسئولیت	فناوری اصلی
نمایش	داشبورد مشتری و تجربه عمومی. بدون دسترسی مستقیم به WordPress یا Agent ها.	Next.js
برنامه/API	احراز هویت، مجوزدهی، تجمیع، هماهنگ‌سازی، REST، Socket.IO و قراردادهای پایدار.	NestJS
Control Plane کسب‌وکار	صفحات مدیریتی، محتوای ویرایشی، فرایندهای پشتیبانی و خدمات، انتخاب ارائه‌دهنده و کنترل‌های عملیاتی.	WordPress + افزونه Unixsee
اجرای عملیاتی	دریافت داده Agent، پایش، ارسال عملیات، ارزیابی هشدار، ترافیک زنده و تولید فعالیت.	NestJS + PostgreSQL
داده زیرساخت	متریک‌های سری زمانی و بررسی‌های Append-only، آماده برای TimescaleDB.	PostgreSQL / TimescaleDB

۴. مالکیت داده و منبع حقیقت

هر موجودیت باید دقیقاً یک مالک مرجع داشته باشد. ارجاع بین سیستمها مجاز است، اما تکثیر کامل و دوطرفه داده مجاز نیست.

دامنه / موجودیت	مالک مرجع	قاعده معماری
فهرست کاربران و پروفایل مشتری	WordPress	NestJS فقط شناسه خارجی کاربر WordPress، عضویت Tenant، نمای مجوزها و داده نشست لازم برای اعمال سیاستهای API را نگهداری می‌کند.
نشستها Access/Refresh Token و	NestJS	NestJS جریان ورود را اعتبارسنجی می‌کند، در صورت نیاز مرحله دوم را انجام می‌دهد، Refresh Token ها را می‌چرخاند و توکنهای Unixsee را صادر می‌کند.
کاتالوگ پلن‌ها	WordPress	NestJS یک Read Model نرمال شده برای Next.js ارائه می‌دهد. قیمت‌ها و عنوان پلن‌ها نباید در فرانت‌اند Hard-code شوند.
درخواست‌های پلن و وضعیت راه‌اندازی	WordPress	این یک فرایند کسب‌وکاری با تمرکز زیاد بر پلن مدیریت است. NestJS فرمان‌ها را Proxy کرده و وضعیت را برای داشبورد نرمال می‌کند.
سرورها، VPS ها، Agent ها، وبسایت‌ها و تخصیص سرویس‌ها	NestJS/PostgreSQL	صفحات مدیریتی WordPress این رکوردها را از طریق API مدیریتی NestJS مدیریت می‌کنند؛ WordPress وضعیت کامل آن‌ها را تکثیر نمی‌کند.
دسترسی‌پذیری، بررسی نرم‌افزار، وضعیت امنیت، ترافیک و متریک‌ها	NestJS/PostgreSQL	این داده‌ها توسط Agent ها و Job های بک‌اند تولید می‌شوند. فیلدهای زنده از طریق Socket.IO ارسال می‌شوند.
فعالیت‌های Unixsee	NestJS/PostgreSQL	یک فید واحد برای فعالیت‌های دستی مدیران و رویدادهای عملیاتی خودکار. WordPress آیتم‌های دستی را از طریق API مدیریتی NestJS ارسال می‌کند.
اعلان‌ها/اخبار داشبورد	WordPress	آیتم‌های محتوایی در WordPress مدیریت و از طریق NestJS تحویل داده می‌شوند.
تیکت‌ها، پیام‌ها و متادیتای فایل‌های پیوست	WordPress	NestJS نمای مقابل کاربر است. فایل‌های باینری خارج از هر دو پایگاه داده برنامه ذخیره می‌شوند.
درخواست‌ها و تخصیص‌های خدمات تکمیلی	WordPress	فرایند کسب‌وکار و بررسی مدیر در WordPress انجام می‌شود؛ NestJS یک API پایدار برای مشتری ارائه می‌دهد.
مرکز راهنما، استوری‌ها، ارزیابی‌ها، عضویت‌ها، نظرات مشتریان و وبلاگ	WordPress	مدیریت محتوا و سرخ‌ها از مسئولیت‌های بومی WordPress است.

دامنه / موجودیت	مالک مرجع	قاعده معماری
jobهای عملیاتی مانند پاک سازی کش و بررسی مجدد وضعیت	NestJS/PostgreSQL	NestJS عملیات ها را مجاز، صف بندی، ارسال، ثبت و ممیزی می کند.
صورتحساب و پرداخت	WordPress/WooCommerce، در فاز بعد	NestJS داده صورتحساب را ارائه می دهد و عملیات پرداخت/سفارش را به یکپارچه سازی WordPress واگذار می کند.

قاعده حیاتی: NestJS نباید مستقیماً جدول های WordPress را Query کند. ارتباط فقط از طریق یک API خصوصی و نسخه بندی شده که توسط افزونه Unixsee در WordPress پیاده سازی شده انجام می شود.

۵. مرز ماژول های NestJS

ماژول	مسئولیت های اصلی
Auth & Identity	هماهنگ سازی ورود، Access/Refresh Token، مرحله دوم احراز هویت، لغو نشست، نگاشت کاربر WordPress و Context مربوط به Tenant.
Dashboard	تجمیع خلاصه وبسایت ها، اعلان ها، فعالیت های Unixsee و شمارنده های مرتبط در یک پاسخ داشبورد.
Plan Requests	نرمال سازی کاتالوگ پلن و وضعیت درخواست؛ واگذاری ساخت درخواست و جریان مدیریتی به WordPress.
Websites & Services	هویت وبسایت، تخصیص پلن، متادیتای سرویس، لینک ها، نمای صورتحساب، مالکیت و مجوزدهی.
Agents & Ingestion	دریافت Payload احراز شده Agent، اعتبارسنجی، Idempotency، ذخیره سازی و انتشار رویداد.
Monitoring & Alerts	وضعیت دسترس پذیری، بررسی نرم افزار/به روزرسانی، وضعیت امنیت، ترافیک، آستانه ها، رخدادها و مدیریت داده قدیمی.
Realtime Gateway	Room های Socket.IO محدود شده به Tenant و به روزرسانی های زنده وبسایت.
Actions	پاک سازی کش، بررسی مجدد وضعیت، فرمان های عملیاتی آینده، وضعیت Retry، Jobها و رویدادهای ممیزی.
Activities	فید فعالیت دستی و خودکار وبسایت؛ فیلتر بر اساس Tenant و وبسایت.
WordPress Integration	Client دارای Type، Retry، Circuit Breaker، تبدیل Schema، مدیریت Webhook و بی اعتبار سازی Cache.

ماژول	مسئولیت‌های اصلی
Tickets & Complementary Services	نمای BFF برای فرایندهای پشتیبانی و خدمات تحت مالکیت WordPress.
Content & Leads	نمای BFF برای مرکز راهنما، اعلان‌ها، استوری‌ها، درخواست ارزیابی، عضویت‌ها، نظرات مشتریان و وبلاگ.
Audit & Administration	فرمان‌های مدیریتی، ردیابی ممیزی تغییرناپذیر، هویت عامل و تاریخچه تغییرات.

۶. جریان‌های اصلی سیستم

۶.۱. درخواست پلن و راه‌اندازی وب‌سایت

1. کاربر یک پلن را در Next.js انتخاب می‌کند. صفحه فعلی شبیه Checkout باید به «درخواست پلن» تغییر نام و متن داده شود و نباید مفهوم پرداخت را القا کند.
2. Next.js درخواست را همراه با پلن انتخاب‌شده و اطلاعات تماس/وب‌سایت به NestJS ارسال می‌کند.
3. NestJS، Tenant را اعتبارسنجی کرده و درخواست را به WordPress Unixsee Bridge منتقل می‌کند.
4. WordPress درخواست را ایجاد کرده و برای ارزیابی و پیگیری در اختیار مدیران قرار می‌دهد.
5. پس از تأیید، مدیران زیرساخت را تأمین یا انتخاب می‌کنند. رکوردهای عملیاتی سرور و وب‌سایت از طریق API مدیریتی NestJS ساخته می‌شوند.
6. فرایند مهاجرت یا ساخت وب‌سایت خارج از رابط مشتری ادامه پیدا می‌کند و نقاط عطف تأییدشده می‌توانند رویداد فعالیت Unixsee ایجاد کنند.
7. زمانی که وب‌سایت فعال و به Tenant کاربر تخصیص داده شود، به‌صورت خودکار در داشبورد نمایش داده می‌شود.

۶.۲. نمای کلی داشبورد

1. Next.js یک مدل نمایشی واحد برای داشبورد از NestJS درخواست می‌کند.
2. NestJS خلاصه وب‌سایت‌ها و فعالیت‌های اخیر را از PostgreSQL می‌خواند.
3. NestJS اعلان‌های منتشرشده را از WordPress، ترجیحاً از Cache کوتاه‌مدت، می‌خواند.
4. NestJS یک پاسخ واحد، فیلترشده بر اساس Tenant و متناسب با کامپوننت‌های پیاده‌سازی‌شده داشبورد باز می‌گرداند.
5. پس از Hydration اولیه REST، فقط فیلدهای زنده از طریق Socket.IO به‌روزرسانی می‌شوند.

۶.۳. جزئیات وبسایت و عملیات‌های اجرایی

داده اولیه صفحه از طریق REST بازگردانده می‌شود. فیلدهای زیر زنده هستند و باید با Socket.IO به‌روزرسانی شوند: `availability`، `lastCheckedAt`، `activeNow` و `activeNowMeasuredAt`. مقادیر تاریخی یا ساختاری همچنان از REST تأمین می‌شوند.

- **Clear Cache: Next.js** فرمان را به NestJS می‌فرستد؛ NestJS مجوز را بررسی می‌کند، یک Action Job می‌سازد، آن را به Agent یا Integration صحیح ارسال می‌کند، نتیجه را ثبت کرده و یک رویداد فعالیت ایجاد می‌کند.
- **Renew Service: Next.js** فرمان را به NestJS می‌فرستد؛ NestJS عملیات صورتحساب را به WordPress/WooCommerce واگذار کرده و نتیجه نرمال‌شده را برمی‌گرداند.
- همه فرمان‌های عملیاتی به Idempotency Key نیاز دارند و باید یک رکورد ممیزی ایجاد کنند.

۶.۴. فعالیت‌های Unixsee

فعالیت‌های Unixsee یک خط زمانی عملیاتی قابل مشاهده برای Tenant هستند، نه یک فید عمومی از نوشته‌های WordPress. هر آیتم باید شامل شناسه، عنوان، زمان، ارجاع به وبسایت در صورت نیاز، عامل/منبع، نوع رویداد، سطح نمایش و متادیتای اختیاری باشد.

- **فعالیت دستی:** مدیر یک آیتم را در پنل مدیریت WordPress ایجاد می‌کند؛ افزونه آن را به API فعالیت‌های NestJS ارسال می‌کند.
- **فعالیت سیستمی:** NestJS پس از عملیات موفق یا رویدادهای عملیاتی مانند تهیه Backup یا پاک‌سازی Cache، آیتم را خودکار ایجاد می‌کند.
- داشبورد آخرین آیتم‌ها را از NestJS می‌خواند. صفحه جزئیات وبسایت می‌تواند همان فید را بر اساس شناسه وبسایت فیلتر کند.
- فعالیت‌ها Append-only هستند. اصلاحات باید با ایجاد رویداد جایگزین/اصلاحی انجام شوند، نه با بازنویسی بی‌صدای تاریخچه.

۶.۵. تیکت‌ها

1. کاربر یک سرویس را انتخاب می‌کند، در صورت نیاز یک وبسایت را انتخاب می‌کند، موضوع و توضیحات را وارد می‌کند و فایل‌های پیوست را بارگذاری می‌کند.
2. NestJS مالکیت و محدودیت فایل را اعتبارسنجی می‌کند، فایل‌ها را با URL‌های امضاشده برای Upload/Download در Object Storage ذخیره می‌کند و تیکت را از طریق WordPress ایجاد می‌کند.
3. WordPress تخصیص مدیر، پاسخ‌ها و جریان وضعیت را مدیریت می‌کند.
4. NestJS فهرست نرمال‌شده تیکت‌ها، Thread جزئیات و به‌روزرسانی وضعیت را به Next.js برمی‌گرداند.

۶.۶. خدمات تکمیلی

1. کاربر درخواست سرویس را از طریق NestJS ثبت می‌کند.
2. WordPress درخواست را ذخیره کرده و جریان بررسی مدیر را فراهم می‌کند.
3. پس از بررسی، مدیر تخصیص سرویس مشتری را همراه با نوع، وب‌سایت، وضعیت، تاریخ‌ها، سهمیه یا پیشرفت پروژه و تاریخچه فعالیت ایجاد می‌کند.
4. NestJS این تخصیص را در داشبورد ارائه داده و دسترسی Tenant را اعمال می‌کند.

۶.۷. پروفایل و تأیید اطلاعات

- خواندن و ویرایش پروفایل از NestJS عبور می‌کند و از طریق API خصوصی روی رکورد کاربر WordPress اعمال می‌شود.
- تأیید ایمیل و شماره تلفن جریان‌های صریح با Challenge‌های دارای انقضا و محدودیت نرخ هستند.
- ارائه‌دهنده فعال SMS می‌تواند از Control Plane پنل WordPress انتخاب شود، اما صدور توکن و تغییر وضعیت‌های تأیید همچنان تحت کنترل NestJS باقی می‌ماند.
- تغییر رمز عبور، نشست‌های Refresh Token مرتبط را لغو می‌کند.
- احراز هویت دومرحله‌ای اختیاری پیش از صدور توکن توسط NestJS اعمال می‌شود.

۶.۸. محتوا، سرخ‌ها و قابلیت‌های عمومی

قابلیت	مالک و الگوی ارائه
مرکز راهنما	WordPress موضوعات و مقالات را مدیریت می‌کند؛ NestJS API‌های خواندن عمومی/مشتری و Cache را ارائه می‌دهد.
استوری‌های تیم	WordPress پروفایل‌ها و آیتم‌های تصویری/ویدیویی استوری را مدیریت می‌کند؛ رسانه‌ها در Media Library وردپرس یا Object Storage/CDN خارجی ذخیره می‌شوند.
ارزیابی فروشگاه	فرم عمومی به NestJS ارسال می‌شود؛ NestJS محافظت در برابر سوءاستفاده را اعمال کرده و رکورد Lead/Request را در WordPress می‌سازد.
عضویت خبرنامه	فرم عمومی از طریق NestJS ارسال می‌شود؛ WordPress رضایت و وضعیت عضویت را ذخیره کرده و با ارائه‌دهنده انتخابی خبرنامه یکپارچه می‌شود.
نظرات مشتریان	فاز بعد؛ WordPress متن، ارتباط با مشتری، رضایت، ویدیو، وضعیت انتشار و ترتیب را مدیریت می‌کند.
وبلاگ	فاز بعد؛ محتوا کاملاً در WordPress نوشته می‌شود و از طریق NestJS یا یک Endpoint کنترل شده فقط خواندنی ارائه می‌شود.

۷. قراردادهای API و رویداد

۷.۱. گروه‌های REST سمت Client

- /auth و /sessions
- /dashboard
- /plans و /plan-requests
- /websites و /websites/:id
- /websites/:id/actions/*
- /activities و /notifications
- /tickets و /tickets/:id/messages
- /complementary-service-requests و /complementary-services
- /profile و /verification
- /help-center
- /public/assessment ، /public/subscriptions ، /public/stories و در آینده Testimonials/Blog

۷.۲. رویدادهای Socket.IO

هدف Payload	رویداد
availability ، lastCheckedAt ، کد اختیاری آخرین بررسی و وضعیت کهنگی داده.	website.availability.updated
activeNow و activeNowMeasuredAt .	website.traffic.updated
پیشرفت و نتیجه پاک‌سازی کش یا سایر عملیات‌های غیرهمزمان وب‌سایت.	website.action.updated
درج زنده و اختیاری یک آیتم تازه قابل مشاهده از فعالیت Unixsee.	activity.created

Client ها فقط پس از احراز هویت به Room های محدودشده به Tenant و وب‌سایت می‌پیوندند. سرور مجوز Room را از توکن و پایگاه داده استخراج می‌کند؛ Client ها هرگز شناسه Tenant دلخواه را انتخاب نمی‌کنند.

۷.۳. قرارداد یکپارچه‌سازی WordPress

- Endpoint های خصوصی و نسخه‌بندی‌شده در افزونه Unixsee Bridge.
- احراز هویت سرور به سرور با درخواست امضا شده یا mTLS و در صورت امکان محدودیت شبکه.
- DTO های پایدار و مستقل از نام جدول های WordPress و Schema افزونه های ثالث.
- Idempotency برای ساخت درخواست، پاسخها، تمیدها و پردازش Webhook.

- Webhook های خروجی WordPress برای وضعیت کاربر، تغییرات تیکت، تغییرات درخواست پلن، تغییرات خدمات تکمیلی و بی اعتبارسازی محتوای منتشرشده.
- Retry با Backoff محدود، Circuit Breaking و مدیریت Dead-letter برای رویدادهای همگامسازی ناموفق.

۸. الزامات امنیت و چندمستاجری

- Next.js هرگز اعتبارنامه مدیر WordPress، دسترسی پایگاه داده، Secret های Agent یا توکن های زیرساخت را دریافت نمی کند.
- هر درخواست مشتری پیش از اجرای هر Repository Call یا فراخوانی WordPress در NestJS به Tenant محدود می شود.
- Access Token ها کوتاه عمر هستند. Refresh Token ها چرخانده می شوند، در حالت ذخیره شده Hash می شوند، قابل لغو هستند و به رکورد نشست/دستگاه متصل می شوند.
- غیرفعال سازی کاربر در WordPress، تغییر نقش، تغییر رمز عبور و حذف حساب باید از طریق همگامسازی مبتنی بر Webhook نشست های NestJS را محدود یا باطل کند.
- دریافت داده Agent از Credential جداگانه هر HMAC، Agent یا روش مشابه برای احراز درخواست، محافظت در برابر Replay با Timestamp/Nonce و اعتبارسنجی Payload استفاده می کند.
- فایل های پیوست تیکت و درخواست دارای محدودیت Content-Type/Size، اسکن بدافزار، Object Storage خصوصی و URL امضا شده هستند.
- عملیات های اجرایی به Capability Check صریح، Rate Limit، Idempotency، ثبت ممیزی و رفتار امن Timeout/Retry نیاز دارند.
- Secret ها خارج از فیلدهای محتوایی WordPress و سورس کد ذخیره می شوند. رابط مدیر می تواند ارائه دهنده را انتخاب کند، اما Secret باید در Secret Store رمزگذاری شده یا تنظیمات محافظت شده زمان اجرا نگهداری شود.
- همه تغییرات مهم مدیریتی و سیستمی باید عامل، عملیات، هدف، زمان، Correlation ID و نتیجه را ثبت کنند.

۹. قابلیت اطمینان، Cache و کارهای غیرهمزمان

- اگر محتوای WordPress موقتاً در دسترس نباشد، NestJS باید داده ناقص اما قابل استفاده داشبورد را بازگرداند؛ داده عملیاتی وبسایت باید همچنان قابل استفاده بماند.
- داده های کم تغییر WordPress مانند پلن ها، اعلان ها، محتوای مرکز راهنما و استوری های منتشرشده Cache شوند. بی اعتبارسازی از طریق Webhook های WordPress انجام شود.
- دسترس پذیری زنده یا ترافیک فعال نباید طوری Cache شود که گویی داده جاری است. زمان اندازه گیری ذخیره و وضعیت کهنگی داده اعلام شود.
- برای ارسال SMS/Email، اسکن فایل پیوست، Retry کردن Webhook وردپرس، عملیات وبسایت و در آینده تطبیق صورتحساب از Background Job استفاده شود.

- در فاز اول می‌توان از یک Instance واحد NestJS همراه با Event Dispatcher انتزاعی استفاده کرد.
- Redis/BullMQ و Adapter ردیس Socket.IO زمانی اضافه شوند که مقیاس افقی یا حجم ارسال آن‌ها را ضروری کند.
- متریک‌ها Append-only باقی می‌مانند، ستون‌های زمان از نوع Timestamp With Time Zone دارند و از کلید مرکب زمان/موجودیت استفاده می‌کنند تا مسیر مهاجرت مستندشده به TimescaleDB حفظ شود.

۱۰. ترتیب پیشنهادی پیاده‌سازی

مرحله	خروجی‌ها	شرط خروج
۱. زیرساخت پایه	ساختار پروژه NestJS، Schema پایگاه داده PostgreSQL، مدل احراز هویت/نشست، مجوزدهی Tenant، اسکلت WordPress Bridge، قواعد API و Audit Log.	کاربر WordPress بتواند وارد شود و نشست Unixsee محدودشده به Tenant دریافت کند.
۲. راه‌اندازی و وب‌سایت‌ها	کاتالوگ/Read Model پلن، درخواست پلن، جریان مدیریتی راه‌اندازی، API مدیریتی سرور/وب‌سایت، فهرست و جزئیات وب‌سایت، دریافت داده Agent و Socket.IO اولیه.	یک درخواست تأییدشده بتواند به وب‌سایتی تبدیل شود که برای کاربر درست با دسترس‌پذیری/ترافیک زنده نمایش داده می‌شود.
۳. ارتباطات داشبورد	تجمع داشبورد، فعالیت‌های Unixsee، اعلان‌های مدیریت‌شده با WordPress، عملیات وب‌سایت و وضعیت عملیات.	رابط فعلی نمای کلی و وب‌سایت به‌طور کامل با API واقعی کار کند.
۴. فرایندهای پشتیبانی	تیکت‌ها، پیام‌ها، فایل‌های پیوست، درخواست و تخصیص خدمات تکمیلی.	کاربران و مدیران بتوانند هر دو فرایند را از ابتدا تا انتها انجام دهند.
۵. حساب و محتوا	ویرایش پروفایل، تأیید اطلاعات، 2FA اختیاری، مرکز راهنما، استوری‌ها، فرم ارزیابی و عضویت خبرنامه.	قابلیت‌های فعلی حساب و محتوا دیگر از Fixture استفاده نکنند.
۶. قابلیت‌های آینده	صورتحساب/پرداخت، نظرات مشتریان، وبلاگ، Redis/BullMQ، TimescaleDB و هشداردهی پیشرفته.	فقط پس از پایدار شدن مالکیت‌ها و قراردادهای فاز اول معرفی شوند.

۱۱. تصمیم‌ها و محدودیت‌های معماری

تصمیم	دلیل
NestJS تنها API مورد استفاده Next.js است.	از وابستگی فرانت‌اند به Schemaهای WordPress جلوگیری می‌کند، احراز هویت و مجوزدهی را یکپارچه می‌سازد و یک قرارداد پایدار ارائه می‌دهد.
WordPress یک پلتفرم مدیریت/کنترل است، نه پایگاه داده عمومی همه‌چیز.	سرعت پیاده‌سازی پلن مدیریت حفظ می‌شود، بدون اینکه بارهای عملیاتی و سری زمانی به WordPress تحمیل شوند.

تصمیم	دلیل
داده عملیاتی وبسایت تحت مالکیت NestJS است.	Agentها، متریک زنده، عملیات، هشدارها و ارتباطات سرورها به بک اند اختصاصی و ذخیره سازی رابطه ای/سری زمانی نیاز دارند.
فرایندهای کسب و کار/محتوا در موارد مناسب تحت مالکیت WordPress هستند.	تیکت، درخواست، انتشار، صورتحساب و محتوا از پنل مدیریت WordPress و Integrationهای آن سود می برند.
فعالیتها تحت مالکیت NestJS هستند.	فید باید کار دستی تیم را با رویدادهای خودکار زیرساخت ترکیب کند و به صورت پایدار به وبسایت و Tenant متصل بماند.
یکپارچه سازی بین سیستمها مبتنی بر API/رویداد است.	خواندن مستقیم پایگاه داده وابستگی شکننده ایجاد می کند و مجوزها و قواعد داخلی سرویس را دور می زند.
جریان Checkout فعلی به جریان درخواست تبدیل می شود.	فرایند واقعی کسب و کار با ارزیابی و تأمین زیرساخت آغاز می شود، نه پرداخت فوری.

۱۲. موارد خارج از هدف فاز اول

- تفکیک به Microserviceها؛ NestJS باید کار را به صورت Modular Monolith آغاز کند.
- ارتباط مستقیم Next.js با WordPress.
- ذخیره متریکهای پرتکرار در WordPress.
- تکنیر کامل رکوردهای کاربر، وبسایت، تیکت یا سرویس در هر دو پایگاه داده.
- فرض اینکه چون یک عملیات در پنل WordPress نمایش داده می شود، پس خود WordPress نیز باید آن را اجرا کند.
- Checkout کامل پرداخت پیش از پایدار شدن جریان راه اندازی و فعال سازی سرویس.
- Redis، TimescaleDB یا چند Instance از NestJS پیش از آنکه مقیاس واقعی آنها را ضروری کند.

۱۳. معیارهای پذیرش معماری

- مرورگر برای داده برنامه و به روزرسانیهای بلا درنگ فقط با NestJS ارتباط برقرار کند.
- هر موجودیت یک مالک مرجع مستند داشته باشد.
- اختلال WordPress فقط قابلیت های تحت مالکیت WordPress را تضعیف کند و وضعیت عملیاتی وبسایت های ذخیره شده در NestJS را پنهان نکند.
- کاربر نتواند با تغییر یک شناسه به وبسایت ها، تیکت ها، فعالیت ها، درخواست ها یا فایل های پیوست Tenant دیگر دسترسی پیدا کند.
- انتخاب پلن یک درخواست ایجاد کند و هرگز پرداخت را به اشتباه تأیید نکند.

- تأمین زیرساخت توسط مدیر، رکوردهای سرور و وبسایت تحت مالکیت NestJS را از طریق API مدیریتی احرازشده ایجاد کند.
- فعالیتهای داشبورد بتوانند هم از مدیران و هم از رویدادهای خودکار سیستم منشأ بگیرند، اما از طریق یک فید واحد NestJS نمایش داده شوند.
- فیلدهای زنده از طریق Roomهای احرازشده Socket.IO بهروزرسانی شوند، درحالی که REST منبع داده اولیه و تاریخی باقی بماند.
- یکپارچهسازی WordPress بدون تغییر کامپوننتهای Next.js یا افشای Schemaهای WordPress برای Client قابل تعویض یا اصلاح باشد.

مبنای مرجع

این پیشنهاد بر پایه موارد زیر تهیه شده است:

- آرشیو سورس فعلی Next.js مربوط به Unixsee و مسیرها/کامپوننتهای پیادهسازی شده داشبورد.
- نیازمندیهای مورد توافق قابلیتها و جریانهای کاری در بحث معماری.
- سند موجود «معماری سیستم و راهبرد داده» شامل دریافت داده Agent، آمادگی PostgreSQL/TimescaleDB، Socket.IO، انتشار رویداد، چندمستاجری و هشدار رخدادها.